



沈阳建筑大学

上机实验报告

课程名称:	编译原理
实验题目:	实验三 LR 类语法分析
专业班级:	计算机 2101 班
姓 名:	张清晨
学 号:	2104230414
日 期:	2024.05.18

【实验目的】

上机目的：

设计并实现一个 LR 语法分析器，根据文法生成 LR 分析表，实现 LR 分析器的状态转移和语法归约操作，并对输入串进行语法分析。

内容：

- (1) 了解常用的自底向上语法分析方法
- (2) 理解 LR 类分析法的基本原理；掌握 LR 分析中总控程序的实现方法；
- (3) 通过设计、编制、调试一个 LR 分析程序，加深对 LR 类分析方法的深入理解，实现对单词序列进行语法检查和结构分析。

【基本原理和上机步骤】

基本原理：

采用自底向上的语法分析方法 LR(k)方法，根据栈中的符号串和向前查看 k 个输入符号，就能唯一确定分析器的动作是移进还是归约，以及用哪个产生式进行归约。

上机步骤：

LR 分析器的结构

一个 LR 分析器实际是一个带先进后出存储器（栈）的确定下推自动机，它由一个输入串、一个下推栈和一个带有分析表的总控程序组成。栈中存放着由“历史”和“展望”材料抽象而来的各种“状态”。任何时候，栈顶的状态都代表了整个的历史和已推测出的展望。为了有助于明确归约手续，我们把已归约出的文法符号串也同时放进栈里。LR 分析器的每一动作都由栈顶状态和当前输入符号所唯一确定。

分析器的任何一次移动都是根据栈顶状态 S_m 和当前输入符号 a_i ，去查看 ACTION 表并执行 ACTION (S_m, a_i) 规定的动作，直至分析成功或失败。

LR 分析表有两个部分：动作部分 ACTION 和状态转换部分 GOTO。

ACTION[S, a]表明当前状态 S 面临输入符号 a 时应该采取的动作：

- 1、移入：将 S, y 的下一个状态 S 以及当前符号入栈。
- 2、归约：对栈顶的符号串按照某个规则进行归约。
- 3、接受：宣布输入符号串为一个句子。
- 4、报错：宣布输入符号串不是句子。

GOTO[S, U]表示当前状态 S 和非终结符号匹配的时候所转换到的下一个状态。

LR 分析表的构造

LR 分析器的工作过程是由总控程序根据分析表,使得分析器构型从一种构型向另一种构型变化的过程。初始构型: $(S_0, a_1a_2\cdots a_n\$)$, S_0 为分析器的初态, $\$$ 为输入串的括号。

分析过程的每步结果可表示为: $(S_0X_1S_1\cdots X_mS_m, a_1a_{i+1}\cdots a_n\$)$ 。

分析器的下一次动作是由栈顶状态 S_m 和当前输入符号 a_i 所唯一确定的, 即: 执行 $\text{ACTION}[S_m, a_i]$ 规定的动作。经执行各种可能的动作后, 分析器的构型可如下变化:

- 1、若 $\text{ACTION}(S_m, a_i) = \text{“移进 S”}$, 则分析器构型变为 $(S_0X_1S_1\cdots X_mS_ma_iS, a_{i+1}\cdots a_n\$)$ 。
- 2、若 $\text{ACTION}(S_m, a_i) = \text{“归约 } A \rightarrow \beta \text{”}$, 则分析器构型变为 $(S_0X_1S_1\cdots X_{m-r}S_{m-r}AS, a_1a_{i+1}\cdots a_n\$)$, 其中 $S = \text{GOTO}(S_{m-r}, A)$, $|\beta| = r$ 。
- 3、若 $\text{ACTION}(S_m, a_i) = \text{“接受”}$, 则分析成功, 正常停止。
- 4、若 $\text{ACTION}(S_m, a_i) = \text{“ERROR”}$, 语法出错, 进行出错处理。

除了这些工作之外, 开始编写代码之前, 我还需要进行 LR 分析表的构造。

【实验结果】

1. 对下列文法, 用 LR (1) 分析法对任意输入的符号串进行分析:

文法:

```
S → E
E → E + T
E → T
T → T * F
T → F
F → (E)
F → i
```

2. 输入文件 test.txt，存储输入字符串



3. 运行结果

保存在 result.txt 中

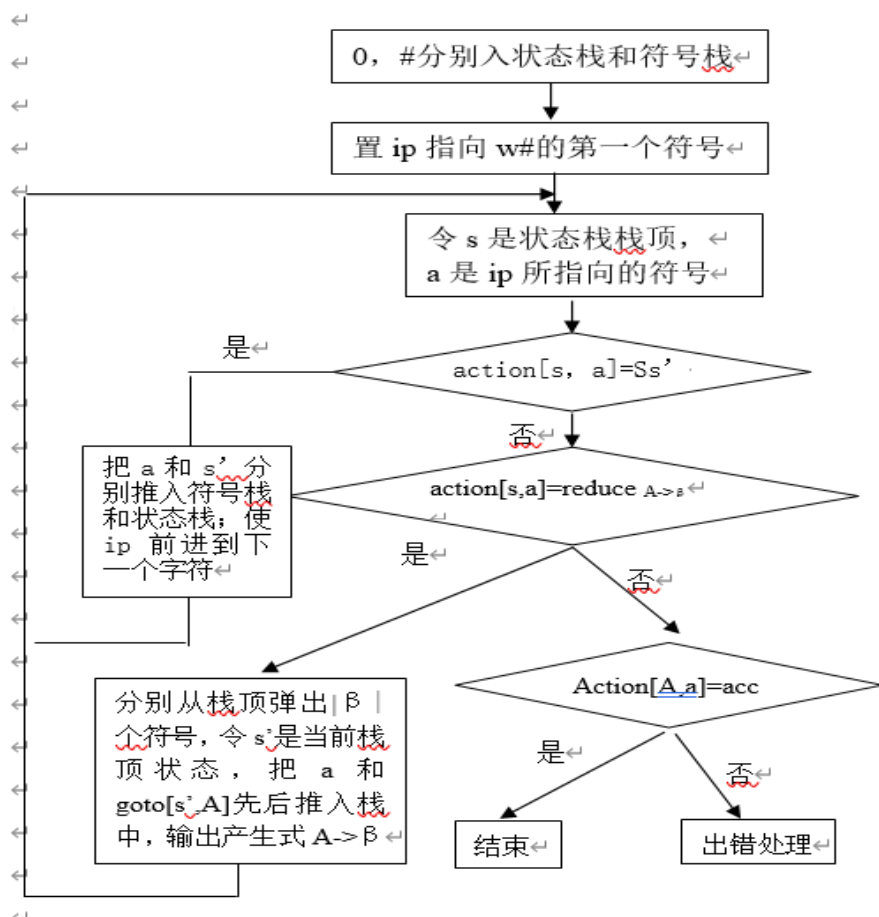
A screenshot of a text editor window titled "result.txt". The window has a menu bar with "文件" (File), "编辑" (Edit), and "查看" (View). The text area contains a table showing the execution results of a parser.

step	状态栈	符号栈	输入符号
0)	0	#	i
1)	05	#i	+
2)	03	#F	+
3)	02	#T	+
4)	01	#E	+
5)	016	#E+	i
6)	0165	#E+i	*
7)	0163	#E+F	*
8)	0169	#E+T	*
9)	01697	#E+T*	i
10)	016975	#E+T*i	#
11)	0169710	#E+T*F	#
12)	0169	#E+T	#
13)	01	#E	#

Acc

【讨论与分析】

实验分析过程：



实验过程记录: 在实验的过程中, 我有很多次错误, 首先我通过自己调试代码的方式解决, 如果未能解决, 又去上网查询资料, 最后都能解决。

实验总结: 在本次 LR 类语法分析实验中, 我深刻体验到了自底向上语法分析的优势和效率。通过逐步构建项目集和识别活前缀, 我掌握了自动生成 LR 分析表的方法, 这极大地加深了我对编译原理中语法分析概念的理解。在实验过程中, 我遇到了状态转换和冲突处理等难题, 但通过耐心调试和深入的理论学习, 我成功地克服了这些障碍, 进一步提高了我的问题解决技巧。同时, 这次实验也锻炼了我的编程能力, 并让我对相关算法和数据结构在实际应用中有了更深的理解。综合来看, 这次实验不仅提升了我的技术能力, 还增强了我对编译器工作原理的深入理解。

【源程序代码】

```
#include <fstream>
#include <iostream>
#include <stdlib.h>
#include <cstring>

struct code_val {
    char code;
    char val[20];
};

const char* p[] = { //产生式
    "S→E","E→E+T","E→T","T→T*F","T→F","F→(E)","F→i"
};

const char TNT[] = "+*()i#ETF"; //LR 分析表列的字符
const int M[][9] = { // LR 分析表数字化，列字符 + *()i#ETF 用数字 012345678 标识。
    { 0, 0, 4, 0, 5, 0, 1, 2, 3}, //0 表示出错，s4 用 4 表示。
    { 6, 0, 0, 0, 0, 99}, //Acc 用 99 表示
    {-2, 7, 0, -2, 0, -2}, //r2 用-2 表示
    {-4, -4, 0, -4, 0, -4},
    { 0, 0, 4, 0, 5, 0, 8, 2, 3},
    {-6, -6, 0, -6, 0, -6},
    { 0, 0, 4, 0, 5, 0, 0, 9, 3},
    { 0, 0, 4, 0, 5, 0, 0, 0, 10},
    { 6, 0, 0, 11},
    {-1, 7, 0, -1, 0, -1},
    {-3, -3, 0, -3, 0, -3},
    {-5, -5, 0, -5, 0, -5}
};

int col(char); //列定位函数原型
using namespace std;

int main()
{
    int state[50] = { 0 }; //状态栈初值
    char symbol[50] = { '#' }; //符号栈初值
    int top = 0; //栈顶指针初值
    ofstream cout("result.txt"); //语法分析结果输出至文件 result.txt
    ifstream cin("test.txt"); // 从 test.txt 中输入词法分析结果
    struct code_val t; //结构变量，存放单词二元式。
```

```

//t.val = "i+i*"#";
cin >> t.code >> t.val; //读一单词
int action;
int i, j = 0; //输出时使用的计数器，并非必要。
cout << "step" << '\t' << "状态栈" << '\t' << "符号栈" << '\t' << "输入符号" <<
endl; //输出标题并非必要。
do {
    cout << j++ << ' ' << '\t'; //输出 step，并非必要。
    for (i = 0; i <= top; i++) cout << state[i]; cout << '\t'; //输出状态栈内容，并非
必要。
    for (i = 0; i <= top; i++) cout << symbol[i]; //输出符号栈内容，并非必要。
    cout << '\t' << t.code << endl; //输出当前输入符号(单词种别)，并非必要。
    action = M[state[top]][col(t.code)];
    if (action > 0 && action != 99) { //移进
        state[++top] = action;
        symbol[top] = t.code;
        t.code = t.val[0];
        for (int x = 0; x < 20; x++) {
            t.val[x] = t.val[x + 1];
        }
        //cin >> t.code >> t.val; //读一单词
    }
    else if (action < 0) { //归约
        if (strcmp(p[-action] + 3, " ε ")) // ε 产生式的右部符号串长度为 0，无
需退栈。
            top = top - (strlen(p[-action]) - 3); //"→"为汉字,占二字节，故减
3。
        state[top + 1] = M[state[top]][col(p[-action][0])]; //产生式左部符号
        symbol[++top] = p[-action][0];
    }
    else if (action == 99) { //接受
        cout << '\t' << "Acc" << endl;
        break;
    }
    else { //出错
        cout << "Err in main()>" << action << endl;
        break;
    }
} while (1);

```

```
}  
int col(char c) //将字符+* ()#ETF 分别转换为数字 012345678  
{  
    for (int i = 0; i < (int)strlen(TNT); i++)  
        if (c == TNT[i])return i;  
    cout << "Err in col char>" << c << endl;  
    exit(0); //终止程序运行  
}
```